## Prerequisites and objectives

- Prerequisites are sets, trees, stacks, queues, priority queues, and induction.
- Choose graph representations, prove search and greedy-edge invariants, and select shortest-path algorithms using their weight conditions.
- Distinguish minimum spanning trees, shortest paths, topological orders, and critical paths by their optimization objectives.

### Graph ADT and problem contract

- A graph is `G=(V,E)`. A directed edge `(u,v)` has a direction. An undirected edge `{u,v}` does not. Weights form a mapping `w:E→R`.
- Unless stated otherwise, graphs have no self-loops or parallel edges. Multigraph extensions need explicit representation and duplicate policies.
- An ADT may enumerate vertices and neighbors, add edges, and query weights. Representation determines each operation's cost.
- Weight 0 is valid. A weighted matrix cannot use the same zero value to mean both a zero-weight edge and no edge.



## Adjacency matrices and lists

| Representation | Space | Test a particular edge | Enumerate neighbors of u |
|---|---|---|---|
| Matrix | $\Theta(V^2)$ | $O(1)$ | $\Theta(V)$ |
| Unsorted lists | $\Theta(V+E)$ | $O(\deg(u))$ | $\Theta(\deg(u))$ |

Here V and E abbreviate set sizes. An undirected edge normally has two adjacency entries. Checking for duplicates before insertion changes the insertion cost.

## The BFS level invariant

1. Set `dist[s]=0` and enqueue s. Other vertices are undiscovered.
2. Dequeue u. For each undiscovered neighbor v, set `dist[v]=dist[u]+1`, `parent[v]=u`, and enqueue v.
3. Mark discovery when enqueuing, preventing duplicate queue entries.

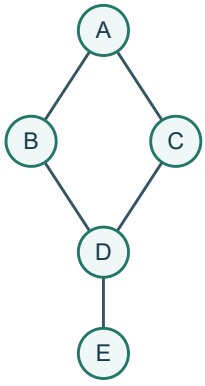
Queued distances are nondecreasing. At v's first discovery, a predecessor on any shorter path would already have been processed. The assigned distance is therefore the minimum edge count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## BFS queue and predecessor trace

### BFS: mark on enqueue and trace the queue




| Dequeued | queue afterward | new parent |
|---|---|---|
| A | [B, C] | B←A, C←A |
| B | [C, D] | D←B |
| C | [D] | — |
| D | [E] | E←D |
| E | [] | — |

Distances A,B,C,D,E = 0,1,1,2,3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## DFS colors, timestamps, and directed cycles

- White vertices are undiscovered, gray vertices are active, and black vertices are finished. The recursion stack is a gray path.
- In a directed graph, a back edge to a gray ancestor proves a directed cycle. An edge to a black vertex alone does not.
- In an acyclic graph, decreasing finish time gives a valid topological order.
- A single-source search may miss other components. Restart at remaining white vertices to construct a search forest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Search cost and reproducible traces

- A complete adjacency-list BFS or DFS handles each vertex and adjacency entry a constant number of times, giving $\Theta(V+E)$ time and $O(V)$ auxiliary space.
- A matrix implementation scans a full row per vertex, giving $\Theta(V^2)$. The list bound cannot be applied without its representation condition.
- Original undirected edges are `AB,AC,BD,CD,DE`. With alphabetical neighbors, BFS from A visits `A,B,C,D,E`.
- Recursive DFS visits `A,B,D,C,E`. Neighbor order changes visit order, but not reachability or BFS distances.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Complete BFS and DFS state machines

```
BFS(s):
    for v in V: dist[v]=infinity; parent[v]=none
    dist[s]=0; Q.push(s)
    while Q not empty:
        u=Q.pop_front()
        for v in adj[u]:
            if dist[v]==infinity:
                dist[v]=dist[u]+1; parent[v]=u; Q.push(v)
DFS(u):
    color[u]=gray; discover[u]=++clock
    for v in adj[u]:
        if color[v]==white: parent[v]=u; DFS(v)
        else if directed and color[v]==gray: report back edge
    color[u]=black; finish[u]=++clock
```

For undirected AB, AC, BD, CD, DE with alphabetical neighbors, queues after processing A, B, C, D, E are [B, C],[C, D],[D],[E],[]. Distances are (0,1,1,2,3), parents (none, A, A, B, D). DFS timestamps are A:(1,10), B:(2,9), D:(3,8), C:(4,5), E:(6,7), with C←D, E←D.



## Complete BFS and DFS state machines (continued)

Gray-target edges detect ancestors in directed DFS. In undirected DFS, ignore the reverse adjacency entry of the parent tree edge. A complete forest initializes all vertices white and restarts at each remaining white vertex. Single-source BFS leaves other components unreachable. Auxiliary worst-case storage is Θ(V), excluding input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## MST objective and the cut property

- A spanning tree of a connected undirected weighted graph has all vertices and V−1 edges. An MST minimizes total weight, allowing negative weights. Disconnected graphs instead have a minimum spanning forest.
- Maintain the invariant that some MST T contains the entire current selected set F. A cut $(S, V \setminus S)$ respects F when no edge of F crosses it. Let e be a lightest crossing edge.

If e∈T, containment already holds. Otherwise adding e to T creates one cycle, containing another edge f crossing the same cut. Since the cut respects F, f∉F. Also w(e)≤w(f), so T′=T−f+e is a spanning tree no heavier than optimal T. It is therefore an MST containing F∪{e}. The exchange preserves every previous selection; this proves safety relative to F.

Individual membership in some MST is insufficient. In a triangle with all three weights 1, each edge belongs to an MST, but the set of all three is a cycle and belongs to none. Safety must refer to the whole chosen set F.



## Prim grows one vertex set

- Let S be the included vertices. For every outside vertex v, store the cheapest edge to S as key[v], with its parent.
- Repeatedly add an outside vertex of minimum key and update its neighbors. The selected edge is lightest across $(S, V \setminus S)$.
- With an indexed binary heap and adjacency lists, time is O((V+E)log V), or O(E log V) for connected nonsingleton graphs. Linear selection with a matrix costs $O(V^2)$.
- Key measures a single edge connecting to the current tree. It is not a cumulative source distance and cannot be substituted for Dijkstra's dist.

Start induction with F=∅. All Prim-selected edges lie inside the current S, so $(S, V \setminus S)$ respects F. Its lightest crossing edge preserves containment in an MST by the safe-edge theorem. In a connected graph the final acyclic F has V−1 edges and is itself that MST.

## Kruskal and disjoint sets

1. Sort all edges by weight.
2. Process `(u,v)` from lightest to heaviest. If endpoints lie in different components, select the edge and union the components.
3. In a connected graph, stop after V−1 selections. Otherwise finish the minimum spanning forest.

Path compression and union by rank give an O(m α(V)) amortized bound for m disjoint-set operations. Sorting normally dominates at O(E log E). Here α is inverse Ackermann, not the hash-table load factor.

For Kruskal, take S as the entire current F-component containing u. A chosen e=(u,v) joins different components, so it crosses a cut respecting F. Any lighter crossing edge would have been processed earlier while its endpoints were still separate; it would have been selected and merged them, a contradiction. Thus e is lightest across this cut, preserving the same containment invariant. Include Θ(V) initialization: total O(V+E log(E+1)+E α(V)) time and Θ(V) DSU space.



## Complete disjoint-set forest operations

A disjoint-set union structure (DSU) represents a partition of vertices. Every nonempty set has one root with parent[root]=root. find returns that root; union merges two different sets and reports whether the partition changed. Allocate V array entries before make_set and require valid vertex IDs.

```
make_set(v): parent[v]=v; rank[v]=0
find(v):
    r=v
    while parent[r]!=r: r=parent[r]
    while parent[v]!=v:
        next=parent[v]; parent[v]=r; v=next
    return r
union(u,v):
    a=find(u); b=find(v)
    if a==b: return false
    if rank[a]<rank[b]: swap(a,b)
    parent[b]=a
    if rank[a]==rank[b]: rank[a]+=1
    return true
```

## Complete disjoint-set forest operations (continued)

The second find loop saves next before overwriting parent; otherwise it loses the uncompressed suffix. Rank records a historical height bound, not the current height after compression, and is never decreased after compression. The Boolean union result is exactly Kruskal's edge-acceptance test.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Disjoint-set states and a self-contained cost bound

Use the same MST's selected edges BC, AB, DE, CD, calling union in that endpoint order. On equal ranks retain the first root. Parent entries below are ordered A,B,C,D,E.

| Completed operation | parent[A..E] | Roots and ranks |
|---|---|---|
| Five make_set calls | [A,B,C,D,E] | A0,B0,C0,D0,E0 |
| union(B,C) | [A,B,B,D,E] | A0,B1,D0,E0 |
| union(A,B) | [B,B,B,D,E] | B1,D0,E0 |
| union(D,E) | [B,B,B,D,D] | B1,D1 |
| union(C,D) | [B,B,B,B,D] | B2 |
| find(E) | [B,B,B,B,B] | B2 |

The final find first follows E→D→B, then links E directly to B. Membership is unchanged. Later edges BD, AC, CE have equal representatives B. Their unions return false without increasing ranks or accepting another edge.

## Disjoint-set states and a self-contained cost bound (continued)

For correctness, induct on the forest invariant. find redirects a node only to its own ancestral root, preserving the partition and acyclicity. union connects two different roots and merges exactly their sets. Rank strictly increases on every nonroot-to-parent edge: a lower rank joins a higher rank; a tie raises the winning root's rank. Compression to an ancestor preserves the inequality. Initially each rank-0 set has one vertex. A rank-r root represents at least $2^r$ vertices, since a rank increase merges two sets of at least $2^{r-1}$ vertices each. Thus $\mathrm{rank} \leq \lfloor \log_2 V \rfloor$, every path and single find/union cost O(1+log(V+1)), initialization costs Θ(V), and storage is Θ(V). This fully proves the usable Kruskal bound O(V+E log(E+1)+E log(V+1)).

The finer inverse-Ackermann layering theorem gives O(V+m α(V)) for V initial make_set calls followed by m find/union operations. The rank-height proof above does not prove that sharper theorem. Neither the implementation nor the proved logarithmic bound depends on the omitted fine-grained analysis. Sorting and initialization of all vertices remain part of the total.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Worked MST trace and counterexample

- Weights are `AB=2,AC=5,BC=1,BD=4,CD=3,CE=7,DE=2`.
- Kruskal can select `BC,AB,DE,CD`, totaling 1+2+2+3=8. Tie order may vary without changing the optimal total.
- Prim from A can select `AB,BC,CD,DE`, the same set in another order.
- An MST does not guarantee shortest source-to-vertex paths. For triangle `AB=2,BC=2,AC=3`, the MST path from A to C costs 4 while the graph's shortest distance is 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

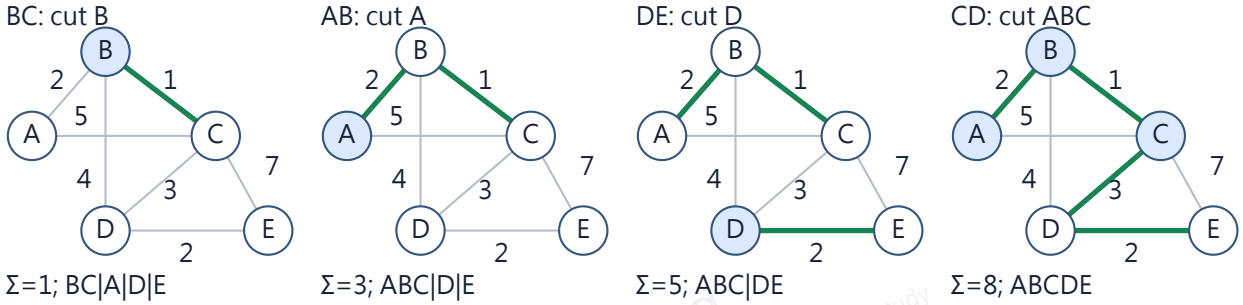
## Complete Prim and Kruskal choices and states

```
Prim(s):
    key[*]=infinity; parent[*]=none; key[s]=0
    while some vertex is unselected:
        u=unselected vertex with minimum key
        if key[u]==infinity: start a new component or report disconnected
        select u
        for (u,v,w) in adj[u]:
            if v unselected and w<key[v]: key[v]=w; parent[v]=u
Kruskal():
    make_set(v) for all v; result=[]
    for (u,v,w) in sorted_edges:
        if find(u)!=find(v): result.append((u,v,w)); union(u,v)
    return result
```



## Complete Prim and Kruskal choices and states (continued)

Kruskal: blue cut side, green selected edges




For AB2, AC5, BC1, BD4, CD3, CE7, DE2, Prim from A gives outside keys after A: B2, C5, D∞, E∞; after B: C1, D4, E∞; after C: D3, E7; after D: E2; then E completes the tree. Parents are B←A, C←B, D←C, E←D, total 8. Key reductions 5→1 and 7→2 are single-edge costs, not accumulated source distances.

Kruskal order is BC1, AB2, DE2, CD3, BD4, AC5, CE7. The first four join different components; the last three would create cycles. DSU establishes component membership, while the cut-exchange theorem proves optimality. Ties may yield multiple distinct minimum trees.

## Shortest paths and relaxation

- A path length sums its edge weights. $\delta(s,v)$ is the minimum source-to-v length when a finite minimum exists. Unreachable vertices have $+\infty$.
- If s reaches a negative cycle that can reach v, walk lengths to v are unbounded below. There is no finite shortest value.
- Relax $(u,v)$ when d[u] is finite and $d[u]+w(u,v)<d[v]$. Update d[v] and its parent.
- Every finite d represents a discovered walk length and is an upper bound on the optimal distance. Machine-integer sums additionally need overflow protection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Dijkstra requires nonnegative weights

- Initialize d[s]=0. Repeatedly select an unsettled u of minimum d, settle it, then relax its outgoing edges.
- The proof requires that the remaining edges of a path cannot lower its accumulated length. All relevant edge weights must be nonnegative.
- In the original counterexample $s\to a=2, s\to b=5, b\to a=-4$, permanently settling a at 2 misses its true distance 1. No negative cycle is needed for this failure.
- Zero-weight edges are legal. Strict positivity is not required. For negative-edge graphs choose an algorithm whose conditions hold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Worked Dijkstra trace and implementation choices

Directed edges are `s→a=4,s→b=1,b→a=2,a→t=1,b→t=6`.

| Settled | d[a] | d[b] | d[t] |
|---|---|---|---|
| s | 4 | 1 | ∞ |
| b | 3 | 1 | 7 |
| a | 3 | 1 | 4 |
| t | 3 | 1 | 4 |

An indexed binary heap gives O((V+E)log V). With duplicate insertions instead of decrease-key, skip stale distances. Space can reach O(V+E), with the conservative time bound O((V+E)log(V+E)).

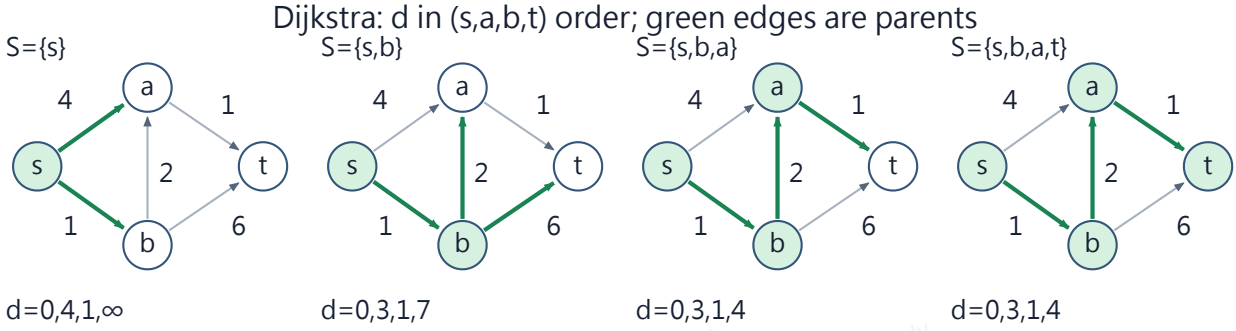
The matrix version scans all unsettled distances to select a minimum and scans the selected row, taking $O(V^2)$ time and $\Theta(V^2)$ matrix space. Adjacency lists with an indexed heap select and update differently. The relaxation rule and nonnegative-weight requirement are shared; their costs are not interchangeable.



## Dijkstra queue entries, stale values, and predecessors

```
Dijkstra(s):
    require all relevant edge weights>=0
    d[*]=infinity; parent[*]=none; d[s]=0
    H.push((0,s)); settled={}
    while H not empty:
        (du,u)=H.pop_min()
        if du!=d[u] or u in settled: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            if d[u]+w<d[v]:
                d[v]=d[u]+w; parent[v]=u; H.push((d[v],v))
```

## Dijkstra queue entries, stale values, and predecessors (continued)




For s→a4, s→b1, b→a2, a→t1, b→t6, start with H=[(0, s)]. Removing s creates (1, b),(4, a). Removing b adds (3, a),(7, t). Removing a=3 adds (4, t). Old (4, a) is skipped because du differs from current d[a]; old (7, t) is also skipped. Final distances are s0, a3, b1, t4 with a←b, b←s, t←a.

If a shorter path to settled u existed, let y be its first unsettled vertex and x its settled predecessor. Relaxation by x makes d[y] no greater than that path prefix. Nonnegative remaining edges make the prefix no longer than the alleged shorter path, contradicting minimum d selection. Negative s→a2, s→b5, b→a−4 breaks that step without a negative cycle. Strict improvement avoids gratuitous equal-distance parent changes that can create cycles around zero-weight edges.



## Bellman–Ford induction on edge count

- Perform V−1 full edge-relaxation passes. In a synchronous-array version, pass k gives shortest walk lengths using at most k edges.
- An in-place version can propagate farther within a pass. After pass k it has nevertheless covered every path of at most k edges.
- Without a reachable negative cycle, a shortest walk can be simplified to a path with at most V−1 edges. A further possible relaxation proves a source-reachable negative cycle.
- In-place relaxation takes O(V+VE) time. The synchronous version here copies V entries per round and takes O(V(V+E)) time, with O(V) distance/parent space. Initialization only at s detects source-reachable negative cycles, not necessarily every negative cycle in the graph.

## Floyd–Warshall recurrence on intermediate vertices

- Without relevant negative cycles, let `D^(k)[i,j]` be the shortest distance whose intermediate vertices belong to `{0,…,k-1}`.
- The recurrence is `D^(k+1)[i,j]=min(D^(k)[i,j],D^(k)[i,k]+D^(k)[k,j])`.
- Initialize diagonal entries to 0, direct edges to their weights, and all other entries to infinity. The k loop must be outermost to match this invariant. Check finiteness before adding distances.
- Time is $\Theta(V^3)$ and space $\Theta(V^2)$. A final `D[v,v]<0` detects a negative cycle. If i reaches such a v and v reaches j, the i-to-j optimum is unbounded below. The computed finite number must not be reported as a finite shortest distance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Bellman–Ford edge layers and Floyd intermediate layers

```
BellmanFord_synchronous(s):
    d[*]=infinity; d[s]=0
    repeat V-1 times:
        next=copy(d)
        for (u,v,w) in E:
            if finite(d[u]): next[v]=min(next[v],d[u]+w)
        d=next
    if any finite(d[u]) and d[u]+w<d[v]: report reachable_negative_cycle
Floyd():
    D=direct_edges_and_zero_diagonal
    for k in V:
        for i in V:
            for j in V:
                if finite(D[i,k]) and finite(D[k,j]):
                    D[i,j]=min(D[i,j],D[i,k]+D[k,j])
```

For s→a2, s→b5, b→a−4, a→t3, synchronous BF vectors (s, a, b, t) are round 0=(0,∞,∞,∞), round 1=(0,2,5,∞), round 2=(0,1,5,5), round 3=(0,1,5,4). An extra pass makes no improvement. In-place relaxation may reach the final vector in its first pass; it does not satisfy the same exact edge-layer equality.

## Bellman–Ford edge layers and Floyd intermediate layers (continued)

For the nonnegative example in order (s, a, b, t), initial Floyd matrix is `[[0,4,1,∞],[∞,0,∞,1],[∞,2,0,6],[∞,∞,∞,0]]`. Intermediate s changes nothing. Intermediate a gives D[s, t]=5, D[b, t]=3. Intermediate b gives D[s, a]=3, D[s, t]=4. Intermediate t changes nothing. Final matrix is `[[0,3,1,4],[∞,0,∞,1],[∞,2,0,3],[∞,∞,∞,0]]`, whose s row matches Dijkstra.

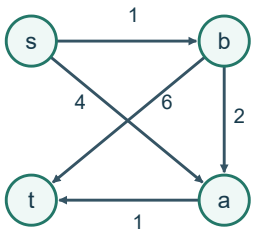
Return to the negative-edge example, whose b→a weight is −4. Adding a→b of weight 1 creates a reachable −3 cycle; adding that edge to the nonnegative example with b→a weight 2 would not. After BF's extra pass, forward search from affected endpoints classifies a, b, t as −∞ while s remains 0. Floyd marks only pairs reaching and leaving a k with D[k, k]<0. A cycle-existence flag is not full affected-vertex classification.

Do not hide copy(d) inside edge-relaxation cost. For a legal graph with $V \geq 2$ and $E=0$, the fixed $V-1$ rounds copy $V(V-1)$ entries, taking $\Theta(V^2)$. The in-place version needs $\Theta(V)$ initialization and $O(V)$ empty edge-loop checks. In general each synchronous round takes $\Theta(V+E)$, so its total is $\Theta(V(V+E))$; the in-place bound is $O(V+VE)$. Both simplify to $O(VE)$ only with an additional $E=\Omega(V)$ assumption. Exact edge-layer states cost two-array maintenance. An early-stop-on-no-change variant terminates after one empty-edge round and must be specified separately.



## Floyd updates as intermediates are admitted

### Floyd–Warshall: adding allowed intermediate vertices




Initially: D[s,a]=4, D[s,b]=1, D[s,t]=∞

k=a: D[s,t]=min(∞,4+1)=5

D[b,t]=min(6,2+1)=3

k=b: D[s,a]=min(4,1+2)=3

D[s,t]=min(5,1+3)=4

Intermediate order s, a, b, t; k is outermost. The final s→b→a→t path has length 4.

The k=a phase establishes b→a→t before k=b uses it to improve s→t.

## Induction for shortest-path recurrences and negative cycles

```
// Synchronous edge-count recurrence:
D[0,s]=0; D[0,v!=s]=infinity
D[k,v]=min(D[k-1,v], min_{(u,v) in E}(D[k-1,u]+w(u,v)))
// Allowed-intermediate recurrence, with k outermost:
F[k+1,i,j]=min(F[k,i,j],F[k,i,k]+F[k,k,j])
```

For synchronous Bellman–Ford, zero edges reach only the source. A walk using at most k edges either already uses fewer edges, or ends in some edge u→v with a prefix using at most k−1 edges. Minimizing over these alternatives proves both directions of the recurrence. When finite shortest distances exist, removing nonnegative cycles yields a simple path with at most V−1 edges, so V−1 rounds suffice.

An improvement in the extra round yields an at-most-V-edge walk better than every at-most-(V−1)-edge walk. It repeats a vertex. If all its cycles were nonnegative, removing a cycle would contradict that improvement; a reachable negative cycle therefore exists. Conversely, if all edges satisfied $d[v] \le d[u]+w(u,v)$, summing these inequalities around a reachable negative cycle would give $0 \le$ a negative number. Thus some edge remains relaxable. Only vertices reachable from affected endpoints have distance $-\infty$; unrelated components remain unaffected.



## Induction for shortest-path recurrences and negative cycles (continued)

Floyd's induction concerns permitted intermediate vertices rather than edge counts. Without a relevant negative cycle, choose a shortest path without repeated vertices. Upon admitting k, it either avoids k or splits into i→k and k→j, each using the old intermediate set. Joining optimal subpaths gives the reverse inequality and proves equality. In-place updates work because D[k,k]=0 leaves D[i,k] and D[k,j] unchanged in this round. With a negative cycle this finite-distance argument fails and reachability must classify $-\infty$ pairs. Three nested loops alone do not justify moving i to the outermost position.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Shortest-path conditions compared

| Method | Condition | Problem and cost |
|---|---|---|
| BFS | Equal positive edge cost, or count edges only | Single source O(V+E) |
| Topological relaxation | Directed acyclic graph, negative edges allowed | Single source O(V+E) |
| Dijkstra | Nonnegative weights | Single source, see “Dijkstra’s priority queue, stale entries, and predecessors” for heap variants |
| Bellman–Ford | Negative edges allowed | Single source; in-place O(V+VE), synchronous O(V(V+E)); detects reachable negative cycles |
| Floyd–Warshall | Negative edges allowed | All pairs $\Theta(V^3)$, classify negative-cycle effects |

Weight restrictions, acyclicity, and single-source versus all-pairs requirements determine the method before data structure preferences.



## Bottleneck paths: objective, relaxation, and full trace

```
bottleneck(s):
    B[*]=infinity; parent[*]=none; B[s]=-infinity
    H.push((-infinity,s)); settled={}
    while H not empty:
        (bu,u)=H.pop_min()
        if u in settled or bu!=B[u]: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            candidate=max(B[u],w)
            if candidate<B[v]:
                B[v]=candidate; parent[v]=u; H.push((candidate,v))
```

A path's bottleneck is its maximum edge weight. The objective is $\min_{P:s\rightsquigarrow t} \max_{e\in P} w(e)$, rather than the sum of its weights. The empty path has max-identity $-\infty$ and unreachable vertices $+\infty$. Nonnegative inputs permit source label 0, but with negative edges that initialization would incorrectly raise an all-negative path's bottleneck to zero.

Use undirected edges s–p=6,p–t=6,s–q=2,q–t=9,p–q=7. After extracting s, B(p,q,t)=(6,2,∞). Extract q to obtain (6,2,9) with t.parent=q. Extract p to obtain (6,2,6) with t.parent=p. Extract t and return bottleneck 6 along s,p,t. The shortest-sum path is instead s,q,t of length 11, while s,p,t totals 12.

## Bottleneck paths: objective, relaxation, and full trace (continued)

For correctness, consider the first unsettled vertex on an allegedly better path. Its settled predecessor has already proposed a value at most that path prefix's bottleneck. Extending a path with max cannot decrease its value, so this candidate is at most the whole better path's bottleneck, contradicting extraction of the minimum-labeled u. This monotonicity holds even with negative edges, unlike additive Dijkstra. Stale-entry bounds match the earlier implementation; indexed binary heaps give O((V+E)log(V+1)).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The MST bottleneck property and threshold connectivity

```
threshold_bottleneck(s,t):
    require undirected graph and s!=t
    make_set(v) for all vertices
    for (u,v,w) in edges sorted by weight:
        union(u,v)
        if find(s)==find(t): return w
    return unreachable
```

In any undirected MST T, let β be the maximum weight on its unique s–t path, and remove a path edge e of weight β. This splits s,t across a cut. A lighter crossing edge f would replace e and strictly reduce tree weight, contradicting minimality. Every s–t path therefore crosses an edge of weight at least β. The tree path achieves β, proving it is minimax.

This is not a shortest-sum property. The example MST is {s–q2,s–p6,p–t6}, total weight 14. Its s–t path totals 12 rather than the graph's shortest 11, but its maximum edge 6 is optimal. Negative edges do not invalidate the exchange or max argument. The tree-path theorem does not transfer directly to directed graphs, where an ordinary MST is not a directed reachability tree.

## The MST bottleneck property and threshold connectivity (continued)

Equivalently, find the smallest threshold $\lambda$ at which $s, t$ become connected in $G_{\leq \lambda}$. Union edges in increasing weight order and return the first connecting weight. Sorting costs O(E log(E+1)); DSU work is amortized O(E α(V)). This solves a single pair's bottleneck, not its additive distance. Repeated queries can use maximum-edge queries on an MST, but that needs separate preprocessing and data-structure analysis.

Initializing all V disjoint sets takes Θ(V) time and Θ(V) DSU space, giving total O(V+E log(E+1)+E α(V)) time. Storing edges for sorting adds a separate Θ(E) edge buffer; sorting workspace depends on the chosen algorithm. With E=0 and V isolated vertices, all sets are still initialized before returning unreachable, taking Θ(V), rather than zero.



## Weight layers and mandatory edges across multiple MSTs

```
classify_MST_edges(G):
    require G undirected and connected
    DSU=singleton components
    for all equal-weight groups in increasing weight:
        Q=multigraph whose vertices are current DSU components
        for edge (u,v) in this group:
            if find(u)==find(v): mark edge as never_in_MST
            else: add its original edge ID between these two vertices in Q
        mark Q bridges as in_every_MST; other Q edges as optional
        for (u,v) in this group: union(u,v)
```

At threshold $\lambda$, let $c_\lambda$ count components of $G_{\leq \lambda}$, the original graph restricted to edges of weight at most $\lambda$. Every MST has exactly $n - c_\lambda$ edges at or below this threshold. Otherwise some threshold component remains disconnected inside the tree's low-weight forest. A low-weight edge reconnects those parts; adding it creates a cycle containing an edge heavier than λ. Replacing that edge strictly lowers tree weight, a contradiction. A forest spanning each threshold component has exactly n−cλ edges.

## Weight layers and mandatory edges across multiple MSTs (continued)

Subtract consecutive threshold counts: every MST selects the same number of edges at each weight, though identities can differ. Alternative edges need not all share one weight globally. Contract strictly lighter components; at the current weight choose a spanning forest in each component of quotient multigraph Q. Bridges must be chosen; cycle edges can be included or omitted; contracted self-loops are never chosen. Classify the entire group before unioning it, or early unions falsely reject equal-weight alternatives.

Q can contain parallel edges even when the original graph is simple. Parallel edges between two components substitute for each other. Bridge DFS must skip the parent edge by ID, not every edge to the parent vertex. Sorting plus linear DFS over all quotient groups costs O(E log(E+1)), with additional amortized DSU work. The result certifies whether an edge is necessary in every optimum, beyond finding one optimum.



## Complete bridge low-link procedure and proof

```
bridges(Q):
    tin[*]=unset; clock=0; result=[]
    for v in Q.vertices:
        if tin[v]==unset: dfs(v,none)
    return result
// dfs shares tin, low, clock, result with the enclosing routine.
dfs(v,parent_edge):
    tin[v]=low[v]=clock; clock+=1
    for (u,edge_id) in Q.adj[v]:
        if edge_id==parent_edge: continue
        if tin[u]==unset:
            dfs(u,edge_id); low[v]=min(low[v],low[u])
            if low[u]>tin[v]: result.append(edge_id)
        else: low[v]=min(low[v],tin[u])
```

Tin is first discovery time. Low[v] is the minimum discovery time reachable by descending zero or more DFS tree edges inside v's subtree and then using at most one nonparent back edge. It is not a shortest-path distance. For tree edge v→u, low[u]>tin[v] means no edge returns from u's subtree to v or an ancestor, so removing the edge disconnects it. If low[u]≤tin[v], a return route bypasses the parent edge and it is not a bridge.

## Complete bridge low-link procedure and proof (continued)

Let Q have X,Y,Z, parallel edges e1,e2 between X,Y, and e3 between Y,Z. Enter Y via e1 with tin(X)=0,tin(Y)=1. Skip e1 at Y, but e2 returns to X and sets low(Y)=0. Visit Z with tin(Z)=low(Z)=2. Since 2>1, e3 is a bridge. Returning to X, low(Y)=0 is not greater than 0, so e1 is not a bridge; neither is back edge e2. Skipping every edge to X would incorrectly classify e1.

Each vertex is discovered once and each undirected adjacency entry is processed once, giving $\Theta(|V_Q|+|E_Q|)$ time and $O(|V_Q|)$ extra space. For MST classification, create quotient vertices only for components incident to the current group. Their total over groups is $O(E)$; allocating a full original-V array for every group would invalidate the claimed total linear group processing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Articulation points: deleting a vertex

An articulation point in an undirected graph is a vertex whose deletion together with incident edges increases the number of connected components. The definition also applies to a disconnected graph; start a DFS tree in each undiscovered component.

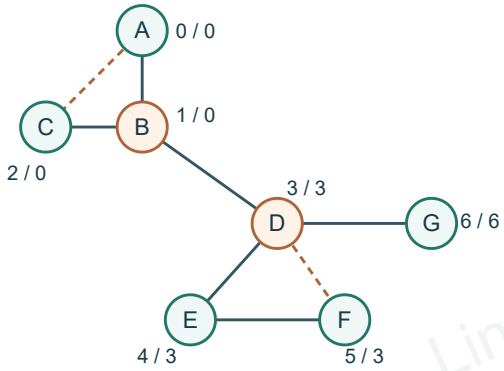
Let tin[v] be discovery time and low[v] the minimum discovery time reachable from v's DFS subtree by zero or more downward tree edges followed by at most one back edge. After finishing tree child u, propagate low[u] into low[v].

A nonroot v is an articulation point exactly when some DFS child u satisfies $\text{low}[u] \geq \text{tin}[v]$. A DFS root instead needs at least two DFS children; it has no parent-side component. A bridge requires $\text{low}[u] > \text{tin}[v]$. The equality case matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A numerical articulation-point and bridge trace

### Articulation points and bridges in one DFS tree




Labels are tin / low

Solid: DFS tree edge

Dashed: back edge to ancestor

B: low[D]=3 ≥ tin[B]=1

D: low[E]=3 ≥ tin[D]=3

Bridges: BD, DG (strict >)

Root A has only one DFS child B, so A is not an articulation point.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Why ≥ for a cut vertex but > for a bridge?

For nonroot v and tree child u, low[u]≥tin[v] means the subtree has no back edge to a strict ancestor of v. Deleting v disconnects that subtree from the parent side. Conversely, low[u] <tin[v] supplies a bypass for each child subtree, so deleting v does not separate that subtree from the parent side.

At equality, a back edge can reach v, so the tree edge (v,u) has an alternative route and is not a bridge. Deleting v removes that return route as well, so v can still be an articulation point. Distinct DFS-root child subtrees cannot have an edge between them, or DFS would have discovered one from the other. At least two such children are therefore required and sufficient.

In the figure low[E]=tin[D]=3: DE is not a bridge, but D is a cut vertex. low[D]=3>tin[B]=1 makes BD a bridge, and low[G]=6>tin[D]=3 makes DG a bridge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Complete articulation-point DFS

```
articulation_points(G):
    tin[*]=unset; clock=0; answer={}
    for v in V:
        if tin[v]==unset: dfs(v, parent_edge=none)
    return answer

dfs(v, parent_edge):
    tin[v]=low[v]=clock; clock+=1; children=0
    for (u, edge_id) in adj[v]:
        if edge_id==parent_edge: continue
        if tin[u]==unset:
            children+=1; dfs(u,edge_id)
            low[v]=min(low[v],low[u])
            if parent_edge!=none and low[u]>=tin[v]: answer.add(v)
        else: low[v]=min(low[v],tin[u])
    if parent_edge==none and children>=2: answer.add(v)
```

The forest includes isolated vertices. Skip the specific incoming edge ID, rather than every edge to the parent vertex, to handle parallel edges. With adjacency lists the time is Θ(V+E), auxiliary space is O(V), and recursion depth can be Θ(V).



## AOV networks and topological sorting

- An activity-on-vertex network represents activities by vertices and precedence by directed edges. A feasible dependency order requires a DAG.
- Kahn's algorithm enqueues all indegree-zero vertices. Output one, remove its outgoing edges, and enqueue any neighbor whose indegree becomes zero.
- Remaining indegrees count only unremoved edges. If vertices remain but none has indegree zero, the remaining graph contains a directed cycle.
- Fewer than V output vertices detects a cycle. Time is Θ(V+E). Multiple eligible vertices allow multiple valid orders.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## AOE networks and earliest event times

- An activity-on-edge network assigns durations to edges and events to vertices. An activity starts only after its preceding event completes.
- Durations d(u, v) are nonnegative. The graph is a DAG with a common start and finish. Add zero-duration dummy edges when there are multiple starts or finishes.
- In topological order, compute `E[v]=max_{(u,v)}(E[u]+d(u,v))`, with E[start]=0.
- This is a longest-path computation on a DAG, not Dijkstra. The maximum expresses the requirement that every prerequisite event must finish.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Latest events, slack, and critical activities

- Set `L[t]=E[t]`, then in reverse topological order compute `L[u]=min_{(u,v)}(L[v]-d(u,v))`.
- Activity `(u,v)` has earliest start E[u] and latest start L[v]−d(u, v). Total slack is their difference.
- Zero-slack activities are critical. Several critical paths may exist. Finding one does not imply all other activities can be delayed.
- The model assumes unlimited resources and fixed durations. Shared workers, machine conflicts, and random durations require further scheduling models.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Worked critical-path data

Activities are `s→a:3,s→b:2,a→t:4,b→t:6`.

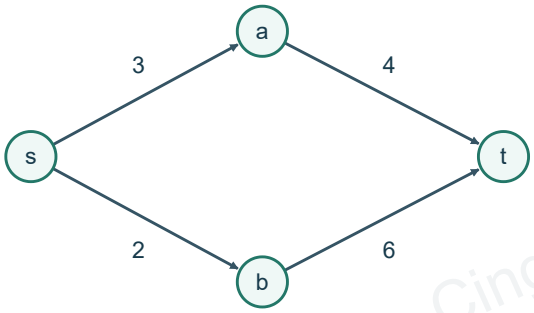
- Earliest events are E[s]=0, E[a]=3, E[b]=2, E[t]=8.
- Latest events are L[t]=8, L[a]=4, L[b]=2, L[s]=0.
- Both s→a and a→t have total slack 1. Both s→b and b→t have slack 0, yielding critical-path duration 8.
- The two noncritical activities cannot both independently consume one extra unit without delaying completion. Slack shared by one path cannot be spent twice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## AOE event times and activity slack

### AOE: event times and activity slack




| Event | earliest E | latest L |
|---|---|---|
| s | 0 | 0 |
| a | 3 | 4 |
| b | 2 | 2 |
| t | 8 | 8 |

s→b→t takes 8; s→a→t takes 7 and shares one unit of slack.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Complete topological and AOE forward/backward computation

```
Kahn():
    indegree=counts of all incoming edges; Q=all zero-indegree vertices
    order=[]
    while Q not empty:
        u=Q.pop(); order.append(u)
        for (u,v) in outgoing: indegree[v]-=1; if indegree[v]==0: Q.push(v)
    require len(order)==V
AOE(order):
    E[*]=-infinity; E[start]=0
    for u in order:
        for (u,v,d): E[v]=max(E[v],E[u]+d)
    L[*]=infinity; L[finish]=E[finish]
    for u in reverse(order):
        for (u,v,d): L[u]=min(L[u],L[v]-d)
```

For s→a3, s→b2, a→t4, b→t6, indegrees start (0,1,1,2). Output s makes a, b zero. Output a lowers t to 1; output b lowers t to 0; output t completes s, a, b, t. Earliest E starts (0,−∞,−∞, −∞), becomes (0,3,2,−∞) after s, t=7 after a, and t=8 after b. Backward L starts with t=8, gives b=2, a=4, then s=min(4−3,2−2)=0.



## Complete topological and AOE forward/backward computation (continued)

Activity tuples (earliest start, latest start, total slack) are s→a:(0,1,1), s→b:(0,0,0), a→t:(3,4,1), b→t:(2,2,0). Fixed durations and unlimited resources give critical path s, b, t of length 8. The length 7 a-path shares one unit of slack; increasing both durations by 1 makes length 9. Add zero-duration virtual boundaries for multiple starts/finishes and restrict analysis to activities reachable between them.

### References and attribution

Lin Cing, handwritten data structures notes; MIT 6.006 lecture notes on graph search and shortest paths.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study